Dockerfile and Docker Registry

Hands-on labs: build images, then publish them

Part 1 shows how a Dockerfile turns instructions into an image, and how RUN, CMD and ENTRYPOINT differ in *when* they execute. Part 2 shows how to share that image through Docker Hub and through your own private registry.

Based on OpenWriteup tutorials: https://www.openwriteup.com/?page_id=825 and https://www.openwriteup.com/?page_id=838

Instruction Cheat Sheet

Instruction	Runs when	Purpose
FROM	Build	Choose the base image
LABEL	Build	Attach metadata such as the author
WORKDIR	Build	Set the working directory for later steps
COPY	Build	Copy a local file into the image
ADD	Build	Like COPY, and can also fetch a URL
RUN	Build	Execute a command and bake the result into the image
EXPOSE	Build (metadata)	Document the port the app listens on
CMD	Container start	Default command; replaced by arguments to docker run
ENTRYPOINT	Container start	Fixed command; docker run arguments are appended

Part 1: Dockerfile

Lab A: Build your first image

Create a working folder, add a Dockerfile that starts from Debian, installs the Apache web server and declares port 80, then build it.

SETUP

```
mkdir doclab
cd doclab
```

DOCKERFILE

```
FROM debian:stable
LABEL authors="amit"
RUN apt-get update && apt-get install -y --force-yes apache2
EXPOSE 80
```

BUILD AND INSPECT

```
docker build -t mydeb .
docker images
docker inspect <imageid>
```

What each line does. FROM picks the base image. LABEL stores metadata. RUN refreshes the package index and installs Apache. EXPOSE records that the service listens on port 80. The **`-t mydeb`** flag names the image and the trailing dot tells Docker to use the current folder as the build context.

> **Caution:** **`--force-yes`** makes apt install packages without asking, even in risky situations. It is not recommended; plain **`-y`** is enough for a normal install.

Lab B: COPY and ADD

This image sets a working directory, copies a local file with COPY and pulls a remote file with ADD.

CREATE A FILE TO COPY

```
touch file.txt
```

DOCKERFILEADD

```
FROM ubuntu:latest
WORKDIR /app
COPY file.txt /app/
ADD https://github.com/amitopenwriteup/my_proj/blob/master/k8s-setup.txt /app/
```

BUILD AND LOOK INSIDE

```
docker build -t myweb2 -f dockerfileadd .
docker images
docker run -it <imageid> /bin/bash
exit
```

-f tells Docker which Dockerfile to use when it is not named *Dockerfile*. Inside the container, check that both files landed in **`/app`**. Prefer COPY for plain local files; reach for ADD only when you need its extras.

RUN vs CMD vs ENTRYPOINT

These three instructions look similar but run at different moments. The four labs below use the same small script so the difference is visible in a log file.

Prepare the script

Create **`date.sh`**. It should append the current date plus any arguments it receives to **`log.txt`**, then print that file. Make it executable.

```
mkdir test
cd test
vi date.sh
chmod 777 date.sh
```

> Using **`chmod +x`** would be tighter than 777, but 777 works for a lab.

Lab 1: RUN executes at build time

RUN happens once, while the image is being created.

MYDOCKERFILE

```
FROM alpine
ADD date.sh /
RUN ["/date.sh", "image created"]
```

```
docker build -t myimage -f mydockerfile .
docker run myimage cat log.txt
```

Result: the log shows one entry, *image created*, and the timestamp is the same every time you run the container, because it was written during the build.

Lab 2: CMD executes at container start

Add a CMD line so the script also runs whenever a container starts.

MYDOCKERFILE1

```
FROM alpine
ADD date.sh /
RUN ["/date.sh", "image created"]
CMD ["/date.sh", "container started"]
```

```
docker build -t myimage1 -f mydockerfile1 .
docker run myimage1
```

Result: the *image created* line stays fixed, while the *container started* line gets a fresh timestamp on every run.

Lab 3: Multiple CMD instructions

What happens if the Dockerfile contains two CMD lines?

MYDOCKERFILE2

```
FROM alpine
ADD date.sh /
RUN ["/date.sh", "image created"]
CMD ["/date.sh", "container started"]
CMD ["/date.sh", "container running"]
```

```
docker build -t myimage2 -f mydockerfile2 .
docker run myimage2
```

> **Rule to remember:** only the **last** CMD in a Dockerfile takes effect, so the output shows *container running* and never *container started*.

Lab 4: ENTRYPOINT

ENTRYPOINT makes the script the fixed program of the container. Anything you type after the image name in docker run becomes its arguments.

MYDOCKERFILE3

```
FROM alpine
ADD date.sh /
RUN ["/date.sh", "image created"]
ENTRYPOINT ["/date.sh"]
```

```
docker build -t myimage3 -f mydockerfile3 .
docker run myimage3 welcome to container
```

Result: the words *welcome to container* are passed to the script and recorded in the log. With CMD those words would have *replaced* the command instead.

Summary: when to use which

	RUN	CMD	ENTRYPOINT
Executes	While building the image	When a container starts	When a container starts
Effect on image	Result is saved in a layer	Stores a default command	Stores the main program
Arguments to docker run	Not applicable	Replace the CMD	Appended to the ENTRYPOINT
Multiple allowed	Yes, each makes a layer	Only the last counts	Only the last counts

Part 2: Docker Registry

A registry is a server that stores images so that other machines can pull them. You can use the public Docker Hub or run your own.

Lab 1: Push an image to Docker Hub

1. Create a free account at hub.docker.com and verify your email if the account is new.

2. Log in from the terminal.

```
docker login
```

3. Tag the local image with your Docker Hub namespace. The lab tags *myimage* as *amitow/myimage*; use your own username. If you leave out a tag, Docker uses **`latest`**.

```
docker tag myimage <username>/myimage
```

4. Push the tagged image.

```
docker image push <username>/myimage
```

> The image then appears in your Docker Hub repositories and can be pulled from any machine with **`docker pull <username>/myimage`**.

Lab 2: Run a private registry

Instead of Docker Hub, host the registry yourself on a server or cloud VM using the official **`registry:2`** image.

1. Start the registry as a container on port 5000 with a restart policy so it comes back after a reboot.

```
docker run -d -p 5000:5000 --restart=always --name registry registry:2
```

2. Tag your image with the registry address.

```
docker tag myimage localhost:5000/my-image
```

3. Push it to the private registry.

```
docker push localhost:5000/my-image
```

4. Verify by asking the registry for its catalog. The reply is a JSON object that lists the image.

```
curl http://localhost:5000/v2/_catalog
```

5. Pull it back. Any Docker host that can reach the registry can now use the image.

```
docker pull localhost:5000/my-image
```

Note: the commands above are the standard forms for these steps; the source page describes them with screenshots. On another machine, replace *localhost* with the registry server's address. A plain-HTTP registry on a non-local host must be listed as an insecure registry in the Docker daemon settings, or protected with TLS.

Registry command summary

Goal	Command
Log in to Docker Hub	docker login
Name an image for a registry	docker tag <image> <registry-or-user>/<name>
Upload an image	docker push <registry-or-user>/<name>
Download an image	docker pull <registry-or-user>/<name>
Start a private registry	docker run -d -p 5000:5000 registry:2
List images in a private registry	curl http://<host>:5000/v2/_catalog